



Guião 2 – Encapsulamento + Herança

Objectivos:

- Atributos privados
- Métodos privados
- Getter/Setter
- Herança
 - Override – redefinição de métodos
- Polimorfismo
 - Classes Abstratas

1. Considere a seguinte classe que armazena um valor de temperatura em Celsius. Instancie um objecto e aceda (ler e escrever) no atributo `temperatura`. Consulte também o valor da temperatura armazenada em Fahrenheit.

```
class Celsius:
    def __init__(self, valor=0):
        self.temperatura = valor

    def to_fahrenheit(self):
        return (self.temperatura * 1.8) + 32
```

2. Considere ainda a seguinte função `soma` (fora da classe). A função tem como argumento de entrada uma lista (tipo `list []`) de objectos `Celsius` e retorna um novo objecto `Celsius` que representa o somatório das temperaturas dos elementos na lista.

```
def soma(lista):
    sum=Celsius(0)
    for t in lista:
        sum.temperatura += t.temperatura
    return sum
```

Crie uma função `def media(lista)` que calcule a média dos valores de temperatura de uma lista de objetos `Celsius`. Deve utilizar a função `soma` no cálculo e retornar a média.



3. Já temos algum código que envolve utilização da classe `Celsius` e que depende directamente do atributo `temperatura`. Suponha que esta classe é mantida por si e que já tem dezenas de utilizadores com código como o anterior a utilizar o `.temperatura`.

Sabendo que a temperatura não pode ser inferior a $-273.15\text{ }^{\circ}\text{C}$, avalie a possibilidade incluir esta verificação.

4. Transforme o atributo `temperatura` em atributo privado. Crie os getters e setters correspondentes de forma a poder manipular a variável privada.

- Adicione ainda a validação anterior no setter. Se menor que -273.15 a temperatura deve ficar nos $-273.15\text{ }^{\circ}\text{C}$.
- E no `__init__`, que fazer?

5. Agora, enquanto cliente e utilizador da classe o código anterior (`soma` e `media`) deixou de funcionar. Altere-o para respeitar as alterações para a nova classe. Qual das versões lhe parece mais legível/limpa (Pythonic).

Obs: Guarde o código anterior, vamos voltar a ele.

6. O atributo `temperatura` (agora `__temperatura`) deixou de estar acessível de “fora”. Crie um `property` com o nome `temperatura` poder manipular o `__temperatura` através do setter e getter anterior. Deverá conseguir executar o código original, feito no ponto 2.

7. A utilização do `property` tem uma versão mais simplificada, utilizando as anotações `@property` e `@temperatura.setter`. Modifique a classe de forma a incluir estas anotações.



7. Adicione à classe um método de objecto (com o `self`) que devolva uma string com o valor da temperatura por extenso.

Ex: `Celsius(-1).extenso()` -> “Menos um grau Celsius”

`Celsius(0).extenso()` -> “Zero graus Celsius”

`Celsius(1).extenso()` -> “Um grau Celsius”

`Celsius(2).extenso()` -> “Dois graus Celsius”

Deve existir um método auxiliar **privado** que apenas retorne a string “grau” ou “graus” de acordo com o valor da temperatura.

Dica:

```
from num2words import num2words
print(num2words(1, lang='pt_BR'))
```

8. Mais alguns testes. Instancie um objecto `Celsius()` e:

- Tente aceder ao atributo (agora property) temperatura.
- Tente aceder ao atributo `__temperatura`.
- Tente outra vez... (dê uma olhadela ao campo `__dict__` do objecto)
- Tente aceder ao método privado criado anteriormente (ponto 7).

O que pode concluir quanto à utilização de atributos/métodos privados

Herança

9. Considere a classe

```
class Pessoa:
    def __init__(self, nome, sobrenome):
        self.nome = nome
        self.sobrenome = sobrenome

    def getNome (self):
        return self.nome + " " + self.sobrenome
```

Crie uma classe `Aluno` que derive de `Pessoa` e inclua um atributo com o número mecanográfico. Inclua um método `getAluno` que retorne uma string com o nome completo e o mecanográfico separados por vírgula.



10. Os métodos `getNome` e `getAluno` estão a retornar uma descrição escrita dos respectivos objetos. Faça o override ao `__str__` por forma a que essa descrição possa ser obtida a partir do `print`.

Ex: `print( Aluno("João", "Silva", 1234) )`

11. Recorrendo à classe `Celsius` do último guião. Vamos refazer a função `soma(lista)`, que devolvia um objeto `Celsius` com a soma das temperaturas na lista. Na classe `Celsius`, implemente o método `__add__` de forma a que seja possível somar dois objetos `Celsius`, diretamente, recorrendo ao operador `+`.

Ex: `Celsius(5) + Celsius(10)`

9. Finalize o guião e envie via moodle.

Encapsulamento

1.

```
1 class Celsius:
2     def __init__(self, valor=0):
3         self.temperatura = valor
4
5     def to_fahrenheit(self):
6         return (self.temperatura * 1.8) + 32
7
8 celsius1 = Celsius(13)
9
10 print(f"Temperatura atual em graus Celsius: {celsius1.temperatura}")
11 print(f"Temperatura atual em graus Fahrenheit: {celsius1.to_fahrenheit():.2f}\n")
12
13 def soma(lista : list[Celsius]) -> Celsius: # retorna um objeto Celsius
14     sum=Celsius(0)
15     for t in lista:
16         sum.temperatura += t.temperatura
17     return sum
18
19 c1 = Celsius(15)
20 c2 = Celsius(20)
21 c3 = Celsius(25)
22 c4 = Celsius(30)
23 c5 = Celsius(35)
24
25 temps = [c1, c2, c3, c4, c5]
26
27 def media(temperaturas : list[Celsius]) -> Celsius:
28     return soma(temperaturas).temperatura / (len(temperaturas))
29
30
31 ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
32 File Edit View Search Terminal Help
33 ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 Celsius.py
34 Temperatura atual em graus Celsius: 13
35 Temperatura atual em graus Fahrenheit: 55.40
```

2.

```
13 def soma(lista : list[Celsius]) -> Celsius: # retorna um objeto Celsius
14     sum=Celsius(0)
15     for t in lista:
16         sum.temperatura += t.temperatura
17     return sum
18
19
20 c1 = Celsius(15)
21 c2 = Celsius(20)
22 c3 = Celsius(25)
23 c4 = Celsius(30)
24 c5 = Celsius(35)
25
26 temps = [c1, c2, c3, c4, c5]
27
28 def media(temperaturas : list[Celsius]) -> Celsius:
29     return soma(temperaturas).temperatura / (len(temperaturas))
30
31
32 ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
33 File Edit View Search Terminal Help
34 ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 Celsius.py
35 Temperatura atual em graus Celsius: 13
36 Temperatura atual em graus Fahrenheit: 55.40
37
38 Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
39 Média de valores da lista de temperaturas em Celsius : 25.0 graus
40 ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 Celsius.py
41 Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
42 Média de valores da lista de temperaturas em Celsius : 25.0 graus
```

3. Adicionaria, por exemplo, uma verificação deste género (linhas 3 a 6) aquando da inicialização de um objeto Celsius, e/ou então poderia adicionar um setter com esta mesma verificação para fazer a validação do valor da temperatura quando quisermos modificá-la.

```
Semana5 > Celsius.py > ...
1 class Celsius:
2     def __init__(self, valor=0):
3         if valor >= - 273.15:
4             self.temperatura = valor
5         else:
6             self.temperatura = -273.15
7
8     def to_fahrenheit(self):
9         return (self.temperatura * 1.8) + 32
10
```

4.

```
Semana5 > Celsius.py > ...
1 class Celsius:
2     def __init__(self, valor=0):
3         self.setTemperatura(valor)
4
5     # getter
6     def getTemperatura(self):
7         return self.__temperatura
8
9     # setter
10    def setTemperatura(self, nova_temp):
11        if nova_temp < - 273.15:
12            self.__temperatura = - 273.15
13        else:
14            self.__temperatura = nova_temp
15
16    # método de instância
17    def to_fahrenheit(self):
18        return (self.__temperatura * 1.8) + 32
19
```

```
Semana5 > ex4.py > ...
1 from Celsius import Celsius
2
3 t = Celsius(10)
4
5 print(t.getTemperatura())
6 t.setTemperatura(-500)
7 print(t.getTemperatura())

ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex4.py
10
-273.15
```

5.

```
Semana5 > ex5.py > soma
1 from Celsius import Celsius
2
3 def soma(lista : list[Celsius]) -> Celsius: # retorna um objeto Celsius
4     sum=Celsius(0)
5     for t in lista:
6         sum.setTemperatura(sum.getTemperatura() + t.getTemperatura())
7     return sum
8
9 c1 = Celsius(15)
10 c2 = Celsius(20)
11 c3 = Celsius(25)
12 c4 = Celsius(30)
13
14 temps = [c1, c2, c3, c4, Celsius(35)]
15
16 def media(temperaturas : list[Celsius]) -> Celsius:
17     return soma(temperaturas).getTemperatura() / (len(temperaturas))
18
19 print(f"\nSomatório das temperaturas dos elementos de uma lista de Celsius : {soma(temps).getTemperatura()} graus")
20 print(f"Média de valores da lista de temperaturas em Celsius : {media(temps)} graus \n")

ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex5.py

Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
Média de valores da lista de temperaturas em Celsius : 25.0 graus
```

A versão anterior é mais legível, mas esta versão protege mais a informação e os atributos de um objeto. Mas, no fundo, ambas as versões são legíveis e fazem o mesmo.

6.

```
1 class Celsius:
2     def __init__(self, valor=0):
3         #self.setTemperatura(valor)
4         self.temperatura = valor
5
6     # @ -> anotações/funções executadas dentro da função // função que mod
7     #intermediário para o setter e o getter; "atributo virtual"
8     #@property
9     def getTemperatura(self):
10         return self.__temperatura
11
12     #@temperatura.setter
13     def setTemperatura(self, nova_temp):
14         if nova_temp < - 273.15:
15             self.__temperatura = - 273.15
16         else:
17             self.__temperatura = nova_temp
18
19     temperatura = property(getTemperatura, setTemperatura)
20
21     # método de instância
22     def to_fahrenheit(self):
23         return (self.__temperatura * 1.8) + 32
24
```

```
Semana5 > ex5.py > soma
1  from Celsius import Celsius
2
3  def soma(lista : list[Celsius]) -> Celsius: # retorna um objeto Celsius
4      sum=Celsius(0)
5      for t in lista:
6          sum.temperatura += t.temperatura
7          #sum.setTemperatura(sum.getTemperatura() + t.getTemperatura())
8      return sum
9
10 c1 = Celsius(15)
11 c2 = Celsius(20)
12 c3 = Celsius(25)
13 c4 = Celsius(30)
14
15 temps = [c1, c2, c3, c4, Celsius(35)]
16
17 def media(temperaturas : list[Celsius]) -> Celsius:
18     return soma(temperaturas).temperatura / (len(temperaturas))
19
20 print(f"\nSomatório das temperaturas dos elementos de uma lista de Celsius : {soma(temps).temperatura} graus")
21 print(f"Média de valores da lista de temperaturas em Celsius : {media(temps)} graus \n")
```

```
ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex5.py
Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
Média de valores da lista de temperaturas em Celsius : 25.0 graus
```

```
Semana5 > ex5.py > soma
1  from Celsius import Celsius
2
3  def soma(lista : list[Celsius]) -> Celsius: # retorna um objeto Celsius
4      sum=Celsius(0)
5      for t in lista:
6          #sum.temperatura += t.temperatura
7          sum.setTemperatura(sum.getTemperatura() + t.getTemperatura())
8      return sum
9
10 c1 = Celsius(15)
11 c2 = Celsius(20)
12 c3 = Celsius(25)
13 c4 = Celsius(30)
14
15 temps = [c1, c2, c3, c4, Celsius(35)]
16
17 def media(temperaturas : list[Celsius]) -> Celsius:
18     return soma(temperaturas).temperatura / (len(temperaturas))
19
20 print(f"\nSomatório das temperaturas dos elementos de uma lista de Celsius : {soma(temps).temperatura} graus")
21 print(f"Média de valores da lista de temperaturas em Celsius : {media(temps)} graus \n")
```

```
ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex5.py
Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
Média de valores da lista de temperaturas em Celsius : 25.0 graus

ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex5.py
Somatório das temperaturas dos elementos de uma lista de Celsius : 125 graus
Média de valores da lista de temperaturas em Celsius : 25.0 graus
```

Usando o property da maneira como indiquei no primeiro print, consigo usar de duas maneiras quando quiser aceder e/ou modificar o atributo temperatura de um objeto (usando apenas o nome "temperatura" da maneira que tinha feito no exercício 2, ou usando o getter e o setter; o resultado foi o mesmo usando das duas formas na função soma).

7.

No ficheiro Celsius.py:

```
27 def extenso(self):
28     return f"{num2words(self.temperatura, lang='pt_BR').capitalize()} {self.__grauString()}"
29
30 def __grauString(self):
31     if( self.temperatura == 1 or self.temperatura == -1 ):
32         return "grau Celsius"
33     else: return "graus Celsius"
```

Se a temperatura do objeto for 1 ou -1, retorna a palavra "grau" do singular, senão retorna "graus". Como o método auxiliar **__grauString** é privado, só se pode usá-lo dentro da classe/ficheiro onde se encontra. Neste caso, usei-o no método **'extenso()'**, no final da string retornada. A função **capitalize()** vai colocar a primeira letra da string em letra maiúscula, como pede o output do exercício.

```
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ sudo apt install python3-num2words
```

No entanto, precisei de instalar o módulo num2words através do apt, para poder importar o módulo e usar o método num2words.

Para testar as funções criadas:

```
Semana5 > ex5.py > ...
10
11 c1 = Celsius(15)
12 c2 = Celsius(20)
13 c3 = Celsius(25)
14 c4 = Celsius(30)
15
16 # temps = [c1, c2, c3, c4, Celsius(35)]
17
18 # def media(temperaturas : list[Celsius]) -> Celsius:
19 #     return soma(temperaturas).temperatura / (len(temperaturas))
20
21 #print(f"\nSomatório das temperaturas dos elementos de uma lista de Celsius")
22 #print(f"Média de valores da lista de temperaturas em Celsius : {media(temps)}")
23
24
25 print(c1.extenso())
26 c2.setTemperatura(-2)
27 print(c2.extenso())
28
29 c3.setTemperatura(-1)
30 print(c3.extenso())
31 c4.setTemperatura(1)
32 print(c4.extenso())
33
34 print(c2.extenso())
35 print(Celsius(-4).extenso())

ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex5.py
Quinze graus Celsius
Menos dois graus Celsius
Menos um grau Celsius
Um grau Celsius
Menos dois graus Celsius
Menos quatro graus Celsius
```


8. O que pode concluir quanto à utilização de atributos/métodos privados?

```
Semana5 > ex8.py > ...
1  from Celsius import Celsius
2
3  # Mais testes:
4
5  temp_celsius = Celsius(21)
6
7  print(f"Aceder ao atributo (agora property) temperatura\nTemperatura: {temp_celsius.temperatura}\n")
8  print(f"Tente aceder ao atributo __temperatura\n __temperatura: {temp_celsius.__temperatura}\n")
9
10
11
ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex8.py
Aceder ao atributo (agora property) temperatura
Temperatura: 21

Traceback (most recent call last):
  File "/home/ines/Documents/PA_ProgramacaoAplicada/Semana5/ex8.py", line 8, in <module>
    print(f"Tente aceder ao atributo __temperatura\n __temperatura: {temp_celsius.__temperatura}\n")
    ~~~~~^~~~~~
AttributeError: 'Celsius' object has no attribute '__temperatura'. Did you mean: 'temperatura'?
```

Foi possível aceder ao atributo (property) temperatura do objeto (temp_celsius). Não foi possível aceder ao atributo privado __temperatura do objeto.

```
Semana5 > ex8.py > ...
1  from Celsius import Celsius
2
3  # Mais testes:
4
5  temp_celsius = Celsius(21)
6
7  print(f"Aceder ao atributo (agora property) temperatura\nTemperatura: {temp_celsius.temperatura}\n")
8  #print(f"Tente aceder ao atributo __temperatura\n __temperatura: {temp_celsius.__temperatura}\n")
9
10 print(f"Tente outra vez... (dê uma olhadela ao campo __dict__ do objecto)\n__dict__ : {temp_celsius.__dict__}\n")
11 print(f"Tente aceder ao método privado criado anteriormente (ponto 7)\n__grauString(): {temp_celsius.__grauString()}\n")
12
13
ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex8.py
Aceder ao atributo (agora property) temperatura
Temperatura: 21

Tente outra vez... (dê uma olhadela ao campo __dict__ do objecto)
__dict__ : {'_Celsius__temperatura': 21}

Traceback (most recent call last):
  File "/home/ines/Documents/PA_ProgramacaoAplicada/Semana5/ex8.py", line 11, in <module>
    print(f"Tente aceder ao método privado criado anteriormente (ponto 7)\n__grauString(): {temp_celsius.__grauString()}\n")
    ~~~~~^~~~~~
AttributeError: 'Celsius' object has no attribute '__grauString'
```

O campo __dict__ do objeto mostra o valor dos atributos do objeto correspondente (neste caso temp_celsius), incluindo os privados. Não foi possível aceder ao método auxiliar privado __grauString da classe Celsius.

Herança

9.

```
Semana5 > Pessoa.py > Pessoa
1 class Pessoa:
2     def __init__(self, nome, sobrenome):
3         self.nome = nome
4         self.sobrenome = sobrenome
5
6     #Getter
7     def getNome (self):
8         return self.nome + " " + self.sobrenome
```

```
Semana5 > Aluno.py > Aluno
1 from Pessoa import Pessoa
2
3 class Aluno(Pessoa):
4     def __init__(self, nome, sobrenome, num):
5         super().__init__(nome, sobrenome)
6         self.numMecanografico = num
7
8     # getter
9     def getAluno(self):
10        return self.getNome() + ", " + self.numMecanografico
```

10.

```
Semana5 > Pessoa.py > Pessoa
1 class Pessoa:
2     def __init__(self, nome, sobrenome):
3         self.nome = nome
4         self.sobrenome = sobrenome
5
6     #Getter
7     def getNome (self):
8         return self.nome + " " + self.sobrenome
9
10    #Override de __str__()
11    def __str__(self):
12        return self.nome + " " + self.sobrenome
```

```
Semana5 > Aluno.py > Aluno > __str__
1 from Pessoa import Pessoa
2
3 class Aluno(Pessoa):
4     def __init__(self, nome, sobrenome, num):
5         super().__init__(nome, sobrenome)
6         self.numMecanografico = num
7
8     # getter
9     def getAluno(self):
10         return self.getNome() + ", " + str(self.numMecanografico)
11
12     def __str__(self):
13         return super().__str__() + ", " + str(self.numMecanografico)
14
15
16 print(Aluno("João", "Silva", 1234))

ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5
File Edit View Search Terminal Help
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 Aluno.py
João Silva, 1234
```

11. Criação da função `__add__` na classe Celsius:

```
Celsius_ex11.py X ex11.py
Semana5 > Celsius_ex11.py > ...
1 from num2words import num2words
2
3 class Celsius:
4     def __init__(self, valor=0):
5         #self.setTemperatura(valor)
6         self.temperatura = valor
7
8     # @ -> anotações/funções executadas dentro da função // função que modifica
9     # intermediário para o setter e o getter; "atributo virtual"
10    #@property
11    def getTemperatura(self):
12        return self.__temperatura
13
14    #@temperatura.setter
15    def setTemperatura(self, nova_temp):
16        if nova_temp < - 273.15:
17            self.__temperatura = - 273.15
18        else:
19            self.__temperatura = nova_temp
20
21    temperatura = property(getTemperatura, setTemperatura)
22
23    # método de instância
24    def to_fahrenheit(self):
25        return (self.__temperatura * 1.8) + 32
26
27    def extenso(self):
28        return f"{num2words(self.temperatura, lang='pt_BR').capitalize()} {self
29
30    def __grauString(self):
31        if( self.temperatura == 1 or self.temperatura == -1 ):
32            return "grau Celsius"
33        else: return "graus Celsius"
34
35    def __add__(self, other_Celsius):
36        return Celsius(self.temperatura + other_Celsius.temperatura)
37
38
39    def __add__(self, other_Celsius):
40        return Celsius(self.temperatura + other_Celsius.temperatura)
```


A função criada faz a soma dos atributos (property) '**temperatura**' de cada objeto Celsius e retorna um novo objeto do tipo Celsius, inicializando a sua temperatura com o resultado da soma feita.

Teste do método `__add__` com o exemplo do enunciado:

```
1 from Celsius_ex11 import Celsius
2
3 print((Celsius(5) + Celsius(10)).extenso())
```

ines@ines: ~/Documents/PA_ProgramacaoAplicada/Semana5

File Edit View Search Terminal Help

```
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana5$ python3 ex11.py
Quinze graus Celsius
```